

Curso de Python

Apuntes de programación

Elaborado por: Lihúén Villarreal

Índice de contenidos

Fundamentos de Python.....	3
Datos y variables.....	3
# VARIABLES.....	3
# DATOS SIMPLES.....	3
# DATOS COMPUESTOS.....	3
Operaciones.....	4
# OPERADORES ARITMETICOS.....	4
# OPERAR CON VARIABLES.....	4
# OPERADORES DE COMPARACIÓN.....	5
# OPERADORES DE PERTENENCIA.....	5
# OPERADORES LOGICOS.....	5
Control de flujo.....	5
# CONDICIONALES IF.....	5
# BUCLES FOR.....	6
# BUCLES WHILE.....	7
Métodos.....	7
# METODOS DE CADENA.....	7
# METODOS DE LISTAS.....	8
# METODOS DE CONJUNTOS.....	9
# METODOS DE DICCIONARIOS.....	9
Funciones.....	10
# FUNCIONES INTEGRADAS.....	10
# DEFINIR FUNCIONES.....	10
# FUNCIONES LAMBDA.....	11
# DECORADORES.....	11
Errores.....	12
# MANEJO DE EXCEPCIONES (ERRORES).....	12
Expresiones regulares.....	13
# EXPRESIONES REGULARES.....	13
Programación Orientada a Objetos.....	14
# CLASES Y OBJETOS.....	14

# CONSTRUCTOR.....	15
# METODOS.....	15
# HERENCIA.....	16
# METODO DE RESOLUCION DE ORDEN (MRO).....	17
# POLIMORFISMO.....	18
# ENCAPSULAMIENTO.....	19
# GETTERS, SETTERS Y DELETTERS.....	19
# DECORADORES PARA GETTERS, SETTERS Y DELETTERS.....	20
# CLASES ABSTRACTAS.....	21
# METODOS ESPECIALES (DUNDER).....	22
# PRINCIPIOS SOLID.....	23
Importar archivos.....	24
# MODULOS.....	24
# PAQUETES.....	25
# ARCHIVOS TXT.....	25
# ARCHIVOS CSV (Valores Separados por Comas).....	25
# EJEMPLOS: TRABAJAR CON ARCHIVOS.....	27
# GRAFICOS.....	27
# LEER ARCHIVOS MUY GRANDES.....	28

Fundamentos de Python

Datos y variables

```
# -----  
# VARIABLES  
# -----  
  
a = 1 # declaro y defino o redefino  
a += 1 # redefino respecto al valor anterior  
del a # borra variable  
# Nombrar variables con snake_case en vez de camelCase (tampoco  
PascalCase, que se usa para clases)  
  
# -----  
# DATOS SIMPLES  
# -----  
  
string = "string en una linea"  
string = 'string en una linea'  
  
"""string  
    en diferentes  
    lineas"""  
  
'''string  
    en diferentes  
    lineas'''  
  
entero = 1 # int (integer)  
decimal = 1.1 # float  
  
True  
False  
# Booleanos se escriben con mayuscula inicial  
  
# -----  
# DATOS COMPUESTOS  
# -----  
  
lista = ['a', 1, 1.1, True]
```

```

lista[0] = 'b' # redefino primer elemento de la lista
lista[1:3] # llamo elementos desde el 1 hasta el 3 (slicing)

tupla = ('a', 1, 1.1, True) # se pueden omitir los parentesis, tupla
de un solo dato lleva coma al final
print(tupla[0])
# Tupla o upla es una lista que no se puede modificar sus elementos
(pero si redefinirse completa)

conjunto = {'a', 1, 1.1, True}
# Set o conjunto es una lista sin orden especifico y no permite
repetir valores
# Se puede llamar completo pero no por elementos

diccionario = {
    'a' : True,
    'b' : 1,
    'c' : 'a',
    'key' : 'value',
    'clave' : 'valor'
}
diccionario['a'] = False
# No se escribe coma despues del ultimo valor del diccionario

```

Operaciones

```

# -----
# OPERADORES ARITMETICOS
# -----

1 + 1
1 - 1
1 * 1
1 / 1 # devuelve un dato tipo float aunque sea entera
1 ** 1 # potencia
1 // 1 # division baja: devuelve la parte entera del resultado
(integer)
1 % 1 # modulo o resto de la division

# -----
# OPERAR CON VARIABLES
# -----

```

```
# Concatenar strings:
nombre = 'Lihuen'
bienvenida = 'Hola ' + nombre + ', bienvenido'
bienvenida = f'Hola {nombre}, bienvenido' # f-strings
```

```
# Desempaquetar arrays:
datos = ['a','b']
a,b = datos # define a='a' y b='b'
```

```
# -----
# OPERADORES DE COMPARACIÓN
# -----
```

```
1 == 1
1 != 1
1 < 1
1 <= 1
1 > 1
1 >= 1
```

```
# -----
# OPERADORES DE PERTENENCIA
# -----
```

```
'a' in string # verifica si se encuentra
'a' not in string # verifica si NO se encuentra
```

```
# -----
# OPERADORES LOGICOS
# -----
```

```
True & False # False
True | False # True
not True # False
```

Control de flujo

```
# -----
# CONDICIONALES IF
# -----
```

```

if 1 == 1:
    print(True)
elif 1 != 1:
    print(False)
else:
    print('WTF?')
# Else if se escribe elif

# -----
# BUCLES FOR
# -----

iterable = [1, 2, 3]
otro_iterable = ['a', 'b', 'c']
string = 'String'
diccionario = {
    'a' : True,
    'b' : 1,
    'c' : 'a',
    'key' : 'value',
    'clave' : 'valor'
}

for i in iterable:
    print(i)
# Itera entre elementos de un iterable

for i,k in zip(iterable, otro_iterable):
    print(i)
    print(k)
# Itera alternando entre dos iterables del mismo tamaño

for i in range(0, 10):
    print(i)
# Itera en un rango numerico, se puede usar range(n) para que itere
n veces

for i in enumerate(iterable):
    indice = i[0]
    elemento = i[1]
# Itera entre tuplas (indice, elemento) a partir del iterable

```

```

for letra in string:
    print(letra)

for i in diccionario.items():
    clave = i[0]
    valor = i[1]
# Itera entre pares (clave, valor) de un diccionario

# For con excepciones:
for i in otro_iterable:
    if i == 'b':
        continue
    # Continue hace que salte a la siguiente iteracion sin ejecutar
    las instrucciones para esta
    if i == 'c':
        break
    # Break corta el bucle (e impide la ejecucion del else si lo
    hubiera)
    print(i)
else:
    print('fin de bucle')
# Else siempre se ejecuta en el for como ultima iteracion (a menos
# que haya un break)

# For en una sola linea de codigo:
iterable_duplicado = [i*2 for i in iterable]

# -----
# BUCLES WHILE
# -----

i = 0
while i < 10:
    print(i)
    i += 1

```

Métodos

```

# -----
# METODOS DE CADENA
# -----

```

```

string = 'String'

string.upper() # convierte todo a mayusculas
string.lower() # convierte todo a minusculas
string.capitalize() # pone solo la primera letra en mayusculas

string.find('i') # encuentra el indice de la primera aparicion del
parametro (da -1 si no encuentra nada)
string.index('i') # igual a find pero si no encuentra nada tira
error
string.count('Str') # devuelve el numero de ocurrencias del
parametro

string.isnumeric() # verifica si es numerico (incluso si es un
string con solo numeros)
string.isalpha() # verifica si es alfanumerico (solo numeros y
letras sin espacios ni caracteres especiales)
string.startswith('S') # verifica si la cadena empieza con el
parametro
string.endswith('g') # verifica si la cadena termina con el
parametro

string.replace('t','p') # reemplaza el primer parametro por el
segundo
string.split('i') # separa usando parametro como separador, devuelve
una lista

# -----
# METODOS DE LISTAS
# -----

lista = ['a', 1, 1.1, True]

lista.append('False') # agrega un elemento al final de la lista
lista.insert(1,'b') # agrega un elemento en el indice indicado
lista.extend(['False', 'b', 2]) # agrega varios elementos al final
de la lista

lista.pop(0) # elimina un elemento en el indice indicado (-1 para
eliminar el ultimo, -2 el anteultimo etc)

```



```

lista.remove('a') # elimina el elemento indicado
lista.clear() # elimina todos los elementos

lista.sort() # ordena de menor a mayor
lista.sort(reverse=True) # ordena de mayor a menor
lista.reverse() # invierte el orden de los elementos

# -----
# METODOS DE CONJUNTOS
# -----

conjunto = {'a', 1, 1.1, True}
otro_conjunto = {'a', 1, 1.1, True, 'b', 2, 2.2, False}

conjunto.issubset(otro_conjunto) # verifica si es subconjunto
(tambien se pueden comparar con <=)
conjunto.issuperset(otro_conjunto) # verifica si es superconjunto
(tambien se pueden comparar con >)
conjunto.isdisjoint(otro_conjunto) # verifica si no hay ningun
elemento en comun

# -----
# METODOS DE DICCIONARIOS
# -----

diccionario = {
    'a' : True,
    'b' : 1,
    'c' : 'a',
    'key' : 'value',
    'clave' : 'valor'
}

diccionario.keys() # devuelve una lista con las claves
diccionario.get('a') # devuelve el valor de la clave indicada, y
None si no la encuentra
diccionario.clear() # elimina el contenido del diccionario
diccionario.pop('a') # elimina el elemento indicado
diccionario.items() # genera una lista iterable con los pares
clave-valor del diccionario

```

Funciones

```
# -----  
# FUNCIONES INTEGRADAS  
# -----
```

```
print(a) # imprime en consola  
type(a) # muestra tipo de dato  
len(a) # length  
dir(a) # devuelve lista de atributos y metodos del objeto  
input('Prompt') # muestra el prompt y devuelve el string que inserte  
el usuario
```

```
int(a) # convierte a numero  
float(a) # convierte a float  
str(a) # convierte a string  
list(a) # crea lista a partir de vector  
tuple(a) # crea tupla a partir de vector  
set(a) # crea conjunto a partir de vector  
frozenset(a) # crea conjunto no modificable a partir de vector  
dict(clave1='valor1', clave2='valor2') # crea diccionario  
dict.fromkeys(['clave1', 'clave2']) # crea diccionario con valores  
vacios a partir de vector de claves
```

```
max(a) # busca el maximo en un vector numerico  
min(a) # busca el minimo en un vector numerico  
sum(a) # suma todos los elementos de un vector numerico  
round(a,2) # redondea un numero (el segundo argumento son las cifras  
decimales, si no se incluye será 0)
```

```
bool(a) # devuelve False si se le pasa 0, datos vacios, False o None  
all(a) # igual a bool pero verifica todos los elementos de un  
iterable  
filter(bool, iterable) # devuelve un iterable con los elementos del  
segundo argumento que verifican el primer argumento
```

```
# -----  
# DEFINIR FUNCIONES  
# -----
```

```
# Definir funcion:  
def funcion(parametros):
```

```

    resultado = len(parametros)
    return resultado

# Llamar funcion:
funcion(a)

# Tomar argumentos y convertirlos en vector:
def funcion(*args):
    return sum(args)

# Funcion con parametro seteado por defecto:
def funcion(parametro = 0):
    return parametro

# Funcion con condicional y bucle:
def es_primo(numero):
    if numero == 2: return False
    for i in range(2, numero-1):
        if numero % i == 0: return False
    return True
# Si ejecuta el return del if, se corta el bucle, si no, ejecuta el
ultimo return

# -----
# FUNCIONES LAMBDA
# -----

resultado = lambda a : round(a)

# -----
# DECORADORES
# -----

# Agregan funcionalidades extra a otras funciones definidas

# Definir decorador:
def decorador(funcion):
    def funcion_extra():
        print('Antes de llamar a la funcion')
        funcion()
        print('Despues de llamar a la funcion')

```

```

        return funcion_extra()

# Utilizarlo renombrando funcion:
def funcion():
    return 'Funcion'

funcion_modificada = decorador(funcion)
funcion_modificada()

# Utilizarlo sin renombrar funcion:
@decorador
def funcion():
    return 'Funcion'

funcion()

```

Errores

```

# -----
# MANEJO DE EXCEPCIONES (ERRORES)
# -----

# Sentencias que pueden ir dentro de la funcion definida:

# Instrucciones de la funcion si todo esta ok:
try:
    print('Instrucciones')
# Instrucciones para el caso de excepcion (error):
except:
    print('Error')
# Instrucciones que se ejecutan despues del try si no hay excepcion:
else:
    print('Instrucciones')
# Instrucciones que se ejecutan independientemente de si se ejecuto
el try o el except:
finally:
    print('Instrucciones')

# Acceder al objeto excepcion:
try:
    print('Instrucciones')
except Exception as e:

```

```

    print(f'Error: {e}')

# Se puede crear una excepcion personalizada como objeto:
class MiExcepcion(Exception):
    def __init__(self, err):
        print(f'Error: {err}')

# Manejar la excepcion personalizada:
try:
    print('Instrucciones')
except:
    raise MiExcepcion('Mensaje de error')

```

Expresiones regulares

```

# -----
# EXPRESIONES REGULARES
# -----

# Modulo oficial de Python para trabajar con expresiones regulares:
import re

texto = '''Lorem ipsum dolor sit amet, consectetur adipiscing elit,
    sed do eiusmod tempor incididunt ut labore et dolore magna
aliqua.
    Ut enim ad minim veniam, quis nostrud exercitation ullamco
laboris
    nisi ut aliquip ex ea commodo consequat.'''

# Buscar coincidencias (todas) sin expresiones regulares:
re.findall('Lorem', texto, flags=re.IGNORECASE)
# Buscar solo la primera coincidencia:
re.search('Lorem', texto)

# Expresiones regulares:
# \d - digitos numericos del (0-9)
# \w - caracteres alfanumericos (a-z, A-Z, 0-9, _)
# \s - espacios en blanco, tabulaciones y saltos de linea
# \n - salto de linea
# . - todo menos salto de linea
# ^ - comienzo de linea
# $ - fin de linea

```

```

# \ - caracteres especiales (escribir el caracter especifico despues
de la \)
# * - comodin (cualquier caracter o ningun caracter)
# % - comodin (cualquier caracter, al menos uno)
# ? - indica como opcional la expresion que le antecede
# {n} - repite n veces la expresion que le antecede
# {n,m} - al menos n, maximo m
# () - agrupar caracteres para aplicar {n} a todo el conjunto
# [] - agrupar caracteres para aplicar {n} a cada caracter del grupo
# | - operador or para buscar una cosa o la otra (si estan ambas
muestra las dos)

# Buscar coincidencias con expresiones regulares:
re.findall(r'\d', texto)
# Buscar TODO MENOS caracteres alfanumericos:
re.findall(r'\W', texto)

# Ejemplo: buscar palabra al comienzo de una linea:
re.findall(r'^Ut', texto, flags=re.M)
# flags=re.M activa la deteccion multi linea

# Ejemplo: buscar fecha DD/MM/AAAA:
re.search(r'\d{2}/\d{2}/\d{4}', texto)

# Ejemplo: buscar una direccion de e-mail:
re.match(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}', texto)

# Ejemplo: buscar direccion web:
re.findall(r'https?://[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}', texto)

# Reemplazar una expresion por otra:
texto = re.sub(r'expresion', 'reemplazo', texto)
# Ejemplo: reemplazar vocales por asteriscos:
texto = re.sub('[aeiou]', '*', texto)

```

Programación Orientada a Objetos

```

# -----
# CLASES Y OBJETOS
# -----

```

```

# Nombrar clases con PascalCase:
class NombreClase():
    propiedad1 = 'valor1'
    propiedad2 = 'valor2'
    propiedad3 = 'valor3'

# Crear objeto (instancia de una clase):
objeto1 = NombreClase()

# Acceder a propiedad (atributo) de objeto:
objeto1.propiedad1

# Averiguar si un objeto es instancia de una clase:
isinstance(objeto1, NombreClase)

# -----
# CONSTRUCTOR
# -----

# Constructor de clase (parentesis opcionales despues del nombre de
# clase):
class NombreClase:
    def __init__(self, atributo1, atributo2):
        self.atributo1 = atributo1
        self.atributo2 = atributo2

# Construir objeto definiendo atributos:
objeto1 = NombreClase('atributo1','atributo2')

# -----
# METODOS
# -----

# Son funciones definidas para una clase

class NombreClase:
    # Metodo constructor:
    def __init__(self, atributo1, atributo2):
        self.atributo1 = atributo1
        self.atributo2 = atributo2
    # Metodo personalizado:

```

```

        def metodo(self):
            print(f'Metodo de {self}')

# Llamar metodo:
objeto1.metodo()

# -----
# HERENCIA
# -----

# Una clase hija hereda atributos y metodos de su clase padre

class ClasePadre:
    def __init__(self, atributo1, atributo2):
        self.atributo1 = atributo1
        self.atributo2 = atributo2

# Crear clase hija:
class ClaseHija(ClasePadre):
    pass

# pass es un statement nulo para dejar algo creado y reemplazar
luego

# Definir atributos heredados, y atributo nuevo:
class ClaseHija(ClasePadre):
    def __init__(self, atributo1, atributo2, atributo3):
        super().__init__(atributo1, atributo2)
        self.atributo3 = atributo3

# Herencia multiple:

class ClasePadre2:
    def __init__(self, atributoA, atributoB):
        self.atributoA = atributoA
        self.atributoB = atributoB

    def metodo_clase2(self):
        print(f'Metodo de {self}')

class ClaseHija(ClasePadre, ClasePadre2):

```



```
def __init__(self, atributo1, atributo2, atributoA, atributoB,
atributo_propio):
```

```
    ClasePadre.__init__(atributo1, atributo2)
```

```
    ClasePadre2.__init__(atributoA, atributoB)
```

```
    self.atributo_propio = atributo_propio
```

```
def metodo_clase2(self):
```

```
    return f'{super().metodo()}'
```

```
# Averiguar si una clase es subclase de otra:
```

```
issubclass(ClaseHija,ClasePadre)
```

```
# -----
```

```
# METODO DE RESOLUCION DE ORDEN (MRO)
```

```
# -----
```

```
# Tenemos una subclase que hereda metodos y atributos de dos clases
(ClaseA y ClaseB).
```

```
# A su vez, cada clase hereda metodos y atributos de dos superclases
respectivamente.
```

```
class SuperclaseA():
```

```
    pass
```

```
class SuperclaseB():
```

```
    pass
```

```
class ClaseA(SuperclaseA):
```

```
    pass
```

```
class ClaseB(SuperclaseB):
```

```
    pass
```

```
class Subclase(ClaseA,ClaseB):
```

```
    pass
```

```
# Si la subclase hereda un metodo o atributo con el mismo nombre que
uno de las clases y superclases,
```

```
# la prioridad de ejecución al llamar al metodo o atributo se
determina por el siguiente orden:
```

```

# 1) Subclase
# 2) ClaseA
# 3) SuperclaseA
# 4) ClaseB
# 5) SuperclaseB

# El orden depende del orden en que se indicaron las clases como
# parametros del constructor.

# Averiguar el MRO de una clase:
NombreClase.mro()

# -----
# POLIMORFISMO
# -----

# Un mismo metodo funciona diferente para cada objeto
# (no es necesario que lo hereden de una clase superior)

class Clase1():
    def metodo(self):
        return 'Resultado1'

class Clase2():
    def metodo(self):
        return 'Resultado2'

objeto1 = Clase1()
objeto2 = Clase2()

objeto1.metodo() # Resultado1
objeto2.metodo() # Resultado2

# Como funcion:

def funcion(objeto):
    return(objeto.metodo())

funcion(objeto1) # Resultado1
funcion(objeto2) # Resultado2

```

```

# -----
# ENCAPSULAMIENTO
# -----

# Hay atributos y metodos "publicos", "protegidos" y "privados" por
convencion:

class Clase:
    def __init__(self):
        self.atributo_publico = 'Valor'
        self._atributo_protegido = 'Valor'
        self.__atributo_privado = 'Valor'

    def metodo_publico(self):
        return 'Resultado'

    def _metodo_protegido(self):
        return 'Resultado'

    def __metodo_privado(self):
        return 'Resultado'

# Se señalizan con guiones bajos

# A los atributos y metodos privados no se puede acceder de forma
normal con la notacion del punto
# Se accede mediante setters y getters

# A los atributos y metodos protegidos se puede acceder mediante el
punto pero se desaconseja
# Se recomienda usar setters y getters

# -----
# GETTERS, SETTERS Y DELETTERS
# -----

# Un getter permite acceder a un atributo
# Un setter permite modificar el valor de un atributo
# Un deleter permite eliminar un atributo

class Clase:

```

```

def __init__(self):
    self.atributo = 'Valor'

# Definir getter:
def get_atributo(self):
    return self.atributo

# Definir setter:
def set_atributo(self, new_atributo):
    self.atributo = new_atributo

# Definir deleter:
def delete_atributo(self):
    del self.atributo

objeto = Clase()

# Llamar al getter:
atributo = objeto.get_atributo()
# Llamar al setter:
objeto.set_atributo('Nuevo valor')
# Llamar al deleter:
objeto.delete_atributo

# -----
# DECORADORES PARA GETTERS, SETTERS Y DELETTERS
# -----

# @property convierte a los getters en propiedades no modificables
# @atributo.setter convierte a los setters en propiedades
modificables
# @atributo.deleter permite eliminar la propiedad

class Clase:
    def __init__(self):
        self.atributo = 'Valor'

    # Getter decorado:
    @property
    def atributo(self):
        return self.atributo

```

```

# Setter decorado:
@atributo.setter
def atributo(self, new_atributo):
    self.atributo = new_atributo

# Deleter decorado:
@atributo.deleter
def atributo(self):
    del self.atributo

objeto = Clase()

# Llamar al getter como propiedad:
atributo = objeto.atributo
# Llamar al setter como propiedad:
objeto.atributo = 'Nuevo valor'
# Llamar al deleter como propiedad:
del objeto.atributo

# -----
# CLASES ABSTRACTAS
# -----

# Abstraccion: ocultar la complejidad de la logica interna de las
funciones
# Clases abstractas: plantillas para crear otras clases, no objetos

# Crear clase abstracta:
from abc import ABC, abstractclassmethod

class ClasePlantilla(ABC):
    @abstractclassmethod
    def __init__(self, atributo):
        self.atributo = atributo

    @abstractclassmethod
    def metodo_abstracto(self):
        pass

    def metodo_generico(self):

```

```

        print(f'{self}')

# Crear clase a partir de plantilla:
class Clase(ClasePlantilla):
    def __init__(self, atributo):
        super().__init__(atributo)

    def metodo_abstracto(self):
        print(f'{self.atributo}')

# Llamar al metodo abstracto y al generico:
Clase.metodo_abstracto()
Clase.metodo_generico()

# -----
# METODOS ESPECIALES (DUNDER)
# -----

# Son metodos integrados para cumplir funciones especiales
# Se señalizan con doble guion bajo al inicio y al final de su
nombre
# Ejemplo: __init__ (metodo constructor)

class Clase:
    # Metodo constructor:
    def __init__(self, atributo1, atributo2):
        self.atributo1 = atributo1
        self.atributo2 = atributo2

    # Metodo para mostrar objeto como string al hacer print(objeto):
    def __str__(self):
        return f'Clase(atributo1={self.atributo1},
atributo2={self.atributo2})'

    # Metodo para mostrar representacion del objeto al hacer
eval(repr(objeto)):
    def __repr__(self):
        return f'Clase({self.atributo1}, {self.atributo2})'

# Muchos metodos especiales permiten sobrecarga de metodos
# Ejemplo: definir lo que pasa al sumar dos objetos de cierta clase:

```

```

# __add__(self, other)

# -----
# PRINCIPIOS SOLID
# -----

# Convenciones y buenas practicas para facilitar el mantenimiento y
escalabilidad del codigo

# S - SRP: Principio de Responsabilidad Unico
# O - OCP: Principio de Abierto/Cerrado
# L - LSP: Principio de Sustitucion de Liskov
# I - ISP: Principio de Segregacion de Interfaz
# D - DIP: Principio de Inversion de Dependencias

# SINGLE RESPONSABILITY PRINCIPLE

# Cada clase tiene que tener una unica responsabilidad o
funcionalidad
# Cada clase debe poder cumplir con su funcionalidad sin depender de
otras

# OPEN/CLOSED PRINCIPLE

# Las entidades tienen que estar abiertas para la extension pero
cerradas para la modificacion
# Se debe poder gregar funcionalidades sin tener que modificar las
ya existentes

# LISKOV'S SUBSTITUTION PRINCIPLE

# Por Barbara Liskov
# Las clases derivadas deben ser sustituibles por sus clases base
# Una clase debe poder utilizarse en todos los lugares donde su
subclase pueda utilizarse

# INTERFACE SEGREGATION PRINCIPLE

# Ningun cliente debe ser forzado a depender de interfaces que no
utilice
# Se debe eliminar las dependencias que no se van a utilizar

```

```
# DEPENDENCY INVERSION PRINCIPLE
```

```
# Los modulos de alto nivel no deben depender de los de bajo nivel,  
ambos deben depender de las abstracciones
```

```
# Las abstracciones no deben depender de los detalles, sino al revés
```

```
# Es decir: las funcionalidades mas basicas no deben depender de las  
mas especificas, sino al revés
```

Importar archivos

```
# -----
```

```
# MODULOS
```

```
# -----
```

```
# Se puede enlazar otros archivos .py (modulos) para utilizar sus  
funciones:
```

```
# import nombre_archivo as nombre_modulo (el as es opcional)
```

```
# Para usar una funcion de ese modulo, se la llama como metodo del  
mismo:
```

```
# nombre_modulo.funcion()
```

```
# Se puede importar solo una funcion/metodo/objeto de un modulo  
(acepta usar as):
```

```
# from nombre_archivo import nombre_objeto
```

```
# Si el modulo se encuentra en otra carpeta del workspace:
```

```
# import nombre_carpeta.nombre_archivo
```

```
# Si el modulo se encuentra en una carpeta externa:
```

```
# import sys
```

```
# sys.path.append('ruta_carpeta')
```

```
# import nombre_archivo
```

```
# -----
```



```

# PAQUETES
# -----

# Los paquetes son carpetas con varios modulos
# Dicha carpeta debe contener un archivo en blanco con nombre
__init__.py para que Python entienda que es un paquete

# Para importar un modulo de un paquete:
# import nombre_paquete.nombre_modulo

# -----
# ARCHIVOS TXT
# -----

# archivo = open('ruta_archivo', encoding='UTF-8')
# leer_archivo_completo = archivo.read()
# leer_archivo_por_lineas = archivo.readlines()
# leer_linea_n = archivo.readline(n)
# archivo.close()

# Cerrar archivo siempre despues de usarlo

# Forma alternativa que cierra archivo automaticamente luego de
ejecutar instrucciones:
# Argumento 'w' habilita permisos de escritura

with open('ruta_archivo', 'w', encoding='UTF-8') as archivo:
    contenido = archivo.read()
    # Agregar una linea al archivo:
    archivo.write('Texto a escribir')
    # Para escribir varias lineas se usa un vector y el separador \n
    al final de cada linea:
    archivo.writelines(['Linea 1\n', 'Linea 2'])

# Con argumento 'a' en el open, el metodo write funciona como append
y agrega sin sobrescribir

# -----
# ARCHIVOS CSV (Valores Separados por Comas)
# -----

```

Importar:

```
import csv
with open('ruta_archivo', encoding='UTF-8') as archivo:
    contenido = csv.reader(archivo)
```

Con Pandas:

```
import pandas as pd
df = pd.read_csv('ruta_archivo', names=['columna 1', 'columna 2']) #
data frame
```

Operar:

```
n = 1
m = 2
```

```
df.describe() # muestra info del data frame
```

```
filas, columnas = df.shape() # devuelve vector con cantidad de filas
y columnas del data frame
```

```
primeras_filas = df.head(n) # primeras n filas
ultimas_filas = df.tail(n) # ultimas n filas
```

```
elemento_especifico = df.loc[n, 'columna']
elemento_especifico = df.iloc[n, m]
```

```
variable1 = df['columna 1']
variable1 = df.loc[:, 'columna 1']
variable1 = df.iloc[:, 1]
```

```
df2 = df.sort_values('columna 1') # ordena el data frame segun los
valores de la columna 1
# Se puede agregar el argumento ascending = False para invertir el
orden
```

```
filtrado = df.loc[df['columna 1']==True, :] # se puede poner
cualquier condicion tanto en filas como columnas
```

```
df = df.concat([df, df2]) # concatena data frames
```

```

# -----
# EJEMPLOS: TRABAJAR CON ARCHIVOS
# -----

# TXT

nombres = ['Juan', 'Pedro', 'Maria', 'Jose']
apellidos = ['Suarez', 'Fernandez', 'Garcia', 'Perez']

with open('registro.txt', 'w') as registro:
    registro.writelines('Personas registradas:\n')
    for i,k in zip(nombres,apellidos):
        registro.writelines(f'Nombre: {i}\nApellido:
{k}\n-----\n')

# CSV

import pandas as pd
df = pd.read_csv('ruta_archivo.csv')

# Convertir a string los datos de una columna:
df['columna 1'] = df['columna 1'].astype(str)
# Reemplazar un dato en una columna:
df['columna 1'].replace('dato_viejo', 'dato_nuevo', inplace=True)

# Eliminar filas con datos faltantes:
df = df.dropna()
# Eliminar columnas con datos faltantes:
df = df.dropna(axis=1)
# Eliminar filas repetidas:
df = df.drop_duplicates()

# Guardar data frame como archivo csv:
df.to_csv('nombre_archivo.csv')

# -----
# GRAFICOS
# -----

import pandas as pd
import matplotlib.pyplot as plt

```

```

import seaborn as sns
df = pd.read_csv('ruta_archivo.csv')

# Grafico de lineas:
sns.lineplot(x='columna 1', y='columna 2', data=df)
# Marcar punto en grafico:
plt.plot('valor x', 'valor y', 'o')

# Grafico de barras:
sns.barplot(x='columna 1', y='columna 2', data=df)

# Grafico de dispersion:
sns.scatterplot(x='columna 1', y='columna 2', data=df)

# Boxplot:
sns.boxplot(x='columna 1', y='columna 2', data=df)

# Mostrar grafico:
plt.show()

# -----
# LEER ARCHIVOS MUY GRANDES
# -----

# Con pandas:
import pandas as pd

def read_csv_in_chunks(file_name):
    for i, chunk in enumerate(pd.read_csv(file_name,
chunksize=1000)):
        print('Chunk # {}'.format(i))
        print(chunk)

read_csv_in_chunks('big_file.csv')

# Con csv:
import csv

def read_csv_in_chunks(file_name):
    with open(file_name, 'r') as f:
        reader = csv.reader(f)

```

```
header = next(reader)
for i, chunk in enumerate(reader):
    print('Chunk # {}'.format(i))
    print(chunk)

read_csv_in_chunks('big_file.csv')
```